

sovereign-shops — Design Document v1

Framework: VORP Core (framework calls isolated in bridge files) **Replaces:** vorp_stores / vorp_shops entirely **Dependencies:** oxmysql, vorp_inventory, vorp_core (via bridge) **Fonts** / **Palette:** Rye + Special Elite, sovereign dark-noir CSS variable set

1. Overview

sovereign-shops is a unified commerce resource that manages two distinct shop classes under one roof: **NPC Stores** (government-run, config-driven, never ownable) and **Player Stores** (purchasable businesses with employees, ledgers, taxes, and full owner management). Both classes share one NUI storefront experience, one database schema family, and one server-side transaction engine, so a buyer walking into any store gets the same polished experience regardless of who runs it.

Design philosophy: no empty stores, no XP gating, everything RP-first. Ownership acquisition happens through roleplay (real estate office / Discord ticket + in-character admin handoff for launch), not through automated purchase menus. The script gives admins a dashboard, not a vending machine.

2. Shop Classes

2.1 NPC Stores

NPC stores are defined entirely in config and are never ownable. Planned store types: General Stores, Fishing Stores, Pelt Traders, Black Market, and Butchers. Each store definition carries its own item catalog (buy list and sell list with prices), and a single store definition can be placed at multiple map locations — one "General Store" config stamped into Valentine, Rhodes, and Blackwater with location-specific coordinates, NPC ped models, headings, and blips.

NPC stores are always manned by an NPC cashier. All revenue from NPC store sales flows into the **government fund**. NPC stores pay no tax and hold no ledger of their own beyond transaction logging into the government fund's history.

Roaming mode: an NPC store can be flagged `(roaming)`. Instead of fixed placements, it carries a pool of possible locations and the server picks one **at server restart** (chosen server-side so every player sees the same spot). Used for the **Black Market** — players have to find where the fence has set up this time. Roaming stores have **no blip, ever**; discovery is pure word-of-mouth RP.

Town exclusion zones: config defines a list of town zones (center coordinate + radius: Valentine, Rhodes, Saint Denis, Blackwater, Strawberry, Annesburg, Van Horn, Tumbleweed, Armadillo, etc.). At restart, the roaming picker filters the location pool against these zones and will never select a spot inside one. Since the pool is hand-curated anyway,

this acts as a safety net — a console warning is logged for any pool entry that sits inside an exclusion zone so it can be cleaned out of the config. Exclusion zones are shared config, reusable by any future roaming store.

2.2 Player Stores

Player stores are created and assigned by admins through the Store Admin Dashboard.

There is no in-script purchase flow at launch — the flow is: Discord property listing → ticket → in-character meeting → admin takes payment in RP → admin assigns ownership, sets the store's three-letter code, purchase price, and tax rate via the dashboard. (A future property/real-estate script will hook into the same assignment exports.)

A player store has: one **owner**, up to one **co-owner**, and up to **five employees**. It is tethered to a register coordinate, carries a customizable storefront identity (name, branding, blip), maintains sellable stock and buy orders, and runs two money ledgers (operating + tax).

Store status: `open`, `closed`, `repossessed`. A closed store hides its catalog — no items shown, nothing sellable, buy orders inactive. Repossessed stores are closed and ownerless, awaiting reassignment ("returned to government").

3. Ownership & Roles

3.1 Roles

Owner — full control, the only role that cannot be transferred by players. Ownership transfer is admin-only, handled through the dashboard when there is a valid RP need.

Co-owner (max 1) — everything the owner can do *except* sell/transfer the store. Implicitly holds all permission flags.

Employees (max 5 by default) — granular permissions granted per-employee by owner/co-owner. Employee cap and co-owner cap are global config values (`Config.MaxEmployees`, `Config.MaxCoOwners`) — server-wide, not per-store tiers.

3.2 Permission Flags

Stored as a bitfield (or JSON array) per employee record:

```
PERM_STOCK      -- add/remove stock, fulfill restocks, manage buy-order
intake
PERM_FUNDS_DEPOSIT -- deposit into operating ledger and tax ledger
PERM_FUNDS_WITHDRAW -- withdraw from operating ledger
PERM_PRICES      -- set item prices, create/end sales
PERM_STOREFRONT  -- edit store name, blip label, branding, open/close
status
```

Hiring, firing, permission editing, wage setting, and webhook configuration are owner/co-owner only. Anyone on staff (any flag) can clock in, use the staff panel, and operate the register.

4. Money Model

4.1 Operating Ledger

Every store transaction lands in the operating ledger as a **typed transaction**:

```
sale          -- customer purchase (+)
purchase      -- buy-order payout to a player (-)
wage          -- employee pay (-)
deposit       -- owner/staff deposit (+)
withdrawal    -- owner/staff withdrawal (-)
tax_transfer  -- movement from operating notes (informational, see below)
adjustment    -- admin correction (dashboard only, logged)
```

The operating balance fluctuates by design — buy orders drain it, sales fill it. Because of that volatility, taxes are **never** pulled from the operating ledger.

4.2 Tax Ledger

A separate deposit-only ledger. Owners/staff (with `PERM_FUNDS_DEPOSIT`) put money in; only the automated tax collection ever takes money out. This guarantees tax money is earmarked and can't be accidentally spent on buy orders or wages.

4.3 Government Fund

Single server-wide balance receiving: all NPC store revenue, all collected property taxes, and the ledgers of repossessed stores. Exposed via export so a future government/treasury script can spend it.

5. Property Tax System

Tax rate is set per-store by admin at assignment time, as a percentage of the store's recorded purchase price, collected **monthly**.

Collection flow (server-side scheduler):

1. On the store's tax due date, attempt auto-deduction of the full tax amount from the tax ledger.
2. **Success** → transfer to government fund, log `tax_collected`, notify owner (webhook + in-game mail).
3. **Failure (insufficient tax ledger)** → immediately generate a Government letter to the owner's P.O. Box stating taxes could not be collected and a 72-hour deadline is in

effect. Store enters `tax_delinquent` state (visible on dashboard).

4. At deadline + 72h, retry the deduction.

5. **Second failure** → automatic repossession: owner and co-owner stripped, employees cleared, store status set to `closed`/`repossessed`, remaining ledger balances swept to government fund, full event logged and fired to admin webhook.

Integration point: letters are delivered through `sovereign_postoffice` via `bridge/mail.lua`.

Tax letters are system-generated Government mail with **no sender P.O. Box**. Required export contract (to be added to `sovereign_postoffice` if it doesn't already support system mail):

```
lua
-- sovereign_postoffice export
exports['sovereign_postoffice']:SendSystemMail({
    recipient = charid,          -- delivered to the character's P.O. Box
    sender     = "Office of the Territorial Government",
    subject    = "Notice of Delinquent Property Tax"
    body       = "...",          -- templated letter text
    replyable  = false,          -- system mail: no return address / no repl
})
```

`sovereign-shops` calls this for: tax collected receipts (optional, config toggle), delinquency notices (72-hour warning), repossession notices, and inactivity warnings at the 30-day flag.

6. Inactivity Repossession

Daily scheduler compares each owner's last character login against thresholds:

- **30 days** — store flagged `inactive_warning` on the admin dashboard.
- **45 days** — automatic repossession (same teardown as tax repossession), ownership returns to government.

Thresholds configurable. Co-owner activity does not reset the clock (owner-based rule), but this is a config toggle in case policy changes.

7. Employees: Clock, Pay, and Presence

7.1 Clock System

Staff clock in/out at the store. Clock-in requires physical presence at the storefront. A server-side presence heartbeat (e.g., every 60s) verifies the clocked-in employee remains within a configurable radius of the register coordinate; leaving the radius for longer than a grace period auto-clocks them out.

7.2 Pay Models

Owner chooses per-employee:

Hourly — pay accrues per verified presence tick while clocked in. No presence, no accrual.

Daily — a flat day rate earned only if the employee clocks in and accumulates a configurable minimum of verified presence time that day (e.g., 60+ minutes). Show up, work, get paid; clock in and wander off, get nothing.

Wages pay out from the operating ledger at clock-out (hourly) or at daily settlement (daily model). If the operating ledger cannot cover a wage, the shortfall is logged as an unpaid-wage entry visible to owner and employee — the script does not create debt or negative balances.

7.3 NPC Cashier Mode

When **no employee is clocked in**, an NPC cashier ped spawns at the register coordinate. The moment someone clocks in, the NPC despawns. The NPC is a presence, not a system of its own — it operates the exact same storefront: the owner's real stock, real prices, active sales, and active buy orders. Every store always has a person behind the counter. Ped model configurable per store.

8. Storefront & Catalog

8.1 Sell Catalog

Owner-managed list of items for sale with per-item price and quantity, backed by store storage. Per-shop categories organize the catalog (categories defined per store, no global XP/reputation gating of any kind).

8.2 Sales & Discounts

Per-item sales defined as a **percentage off with a timer**: staff with `PERM_PRICES` sets discount % and end time; the storefront shows original price struck through, sale price, and remaining time. Sales auto-expire server-side.

8.3 Buy Orders

The store's "we're buying" board: per-item entries with price willing to pay and quantity wanted. When a player sells to a buy order, payment comes from the operating ledger, the order's remaining quantity decrements, and the items land in **store storage** (not directly on shelves) for staff to shelve, price, or use. Orders auto-pause when the operating ledger can't cover a payout.

8.4 Open/Closed

Staff with `PERM_STOREFRONT` toggle the store open or closed. Closed = catalog hidden, purchases and buy orders disabled, storefront UI shows a closed notice (customizable

message, e.g., hours or "back soon").

8.5 Branding & Identity

Per-store customization surfaced in the buyer-facing NUI: store display name, accent color selection, header/tagline text, and blip name. Blip label updates live when the name changes. Branding options are curated (palette-safe accent choices, preset motif/icon set) so every storefront stays inside the sovereign noir design language — modern and beautiful, never over-designed, and no arbitrary image uploads to moderate.

8.6 Blips

Player stores: blip with owner-set name, style reflecting store category, hidden or dimmed while `closed` (config choice). NPC stores: config-defined blips per location.

9. Metadata & Weapon Serials

The transaction engine is metadata-aware end to end: items move with their metadata intact (durability/decay, custom labels, descriptions), and the storefront displays condition to the buyer before purchase.

Weapon serials: when a weapon is sold by a player store, its serial is stamped in the format `CODE-XXXXXX` — the store's three-letter code (set by admin at assignment, immutable by players) followed by a random six-digit number. Serial uniqueness is enforced globally via a serial registry table with a unique constraint (generate → insert → retry on collision). This makes every weapon on the server traceable to its selling storefront — a built-in gift to law RP and gunrunning storylines.

10. Staff Panel

An in-store panel (behind the counter / back room prompt) for anyone on staff:

- Shared notes board (shift notes, messages between staff)
- Shopping/restock lists (wanted items and quantities; staff check items off as stocked)
- At-a-glance: current clock status, today's accrued pay, low-stock warnings, active buy orders

Notes are per-store, timestamped, author-attributed.

11. Logging & Webhooks

Two webhook layers:

Store webhook (owner-configurable): owner sets their own Discord webhook URL in store management. Fires on sales, buy-order purchases, stock changes, hires/fires, deposits/withdrawals, clock events. Owner toggles which event types fire.

Admin webhook (server config): everything above across all stores, plus tax events, repossessions, inactivity flags, and admin dashboard actions.

All events are also written to the database (ledger + event log) regardless of webhook configuration — webhooks are a mirror, never the source of truth.

12. Store Admin Dashboard

A dedicated NUI (command/keybind, admin-gated via bridge permission check) — effectively a commerce MDT in the sovereign design language:

Store directory — all player stores with status badges (open / closed / tax_delinquent / inactive_warning / repossessed), owner name, last owner login.

Store detail view — full stock listing, employee roster with permissions, complete operating + tax ledger history, event log, current buy orders.

Ownership management — assign owner/co-owner, set/change the three-letter code (admin-only), record purchase price, force-transfer, manual repossession.

Tax administration — configure per-store tax rate, view tax ledger balances, outstanding/delinquent taxes across the server, collection history, government fund balance.

Analytics — per-store and server-wide: sales volume over time, top items, wage spend, tax collected, buy-order activity.

Inactivity monitor — the 30/45-day pipeline: flagged owners, days remaining, manual override (extend/exempt for approved absences).

13. Data Model (draft)

```
sql
```

```

sovereign_shops
  id, code CHAR(3) UNIQUE NULL,          -- NULL for NPC stores
  class ENUM('npc','player'),
  name, category,
  owner_charid NULL, coowner_charid NULL,
  status ENUM('open','closed','repossessed'),
  purchase_price, tax_rate, tax_due_date, tax_state,
  branding JSON,                        -- accent, tagline, motif, closed_message
  webhook_url NULL, webhook_events JSON,
  register_coords JSON, npc_model,
  created_at

sovereign_shop_locations                -- NPC multi-placement / roaming pool
  id, shop_id, coords JSON, heading, npc_model, blip JSON,
  is_active                            -- roaming stores: server marks the current

sovereign_shop_employees
  id, shop_id, charid, permissions INT,
  pay_model ENUM('hourly','daily'), pay_rate,
  hired_at, hired_by

sovereign_shop_stock
  id, shop_id, item, quantity, price,
  sale_percent NULL, sale_ends_at NULL,
  category, metadata JSON

sovereign_shop_storage                  -- back-room storage incl. buy-order inventory
  id, shop_id, item, quantity, metadata JSON, source ENUM('stock_pull','buy_order')

sovereign_shop_buy_orders
  id, shop_id, item, unit_price, qty_wanted, qty_filled, active

sovereign_shop_ledger
  id, shop_id, account ENUM('operating','tax'),
  type,                                -- sale|purchase|wage|deposit|withdrawal
  amount, balance_after,
  actor_charid NULL, item NULL, qty NULL, note NULL, created_at

sovereign_shop_timeclock
  id, shop_id, charid, clock_in, clock_out NULL,
  verified_minutes, paid, pay_amount

sovereign_shop_notes

```



```

    id, shop_id, charid, kind ENUM('note','restock'),
    content, checked, created_at

sovereign_weapon_serials
    serial VARCHAR UNIQUE, shop_id, weapon, sold_to_charid, created_at

sovereign_government_fund
    id, type, -- npc_sale|tax|repossession_sweep|spen
    amount, balance_after, ref_shop_id NULL, created_at

```

14. Resource Architecture

```

sovereign-shops/
  fxmanifest.lua
  config/
    config.lua -- global settings, tax scheduler, presence radius,
limits
  npc_stores.lua -- NPC store catalogs + placements
  bridge/
    framework.lua -- vorp core: char id, money, admin check
    inventory.lua -- vorp_inventory: items, metadata, weapons
    mail.lua -- P.O. Box letter delivery
    notify.lua
  server/
    main.lua, transactions.lua, ledger.lua, tax.lua,
    employees.lua, timeclock.lua, serials.lua,
    inactivity.lua, admin.lua, webhooks.lua
  client/
    main.lua, npcs.lua, blips.lua, prompts.lua, nui.lua
  ui/ -- storefront + management + staff panel + admin
dashboard

```

All VORP-specific calls live in `bridge/`, consistent with the sovereign- resource convention.

15. Build Phases

Phase 1 — Foundation & NPC Stores. Schema, bridge layer, NPC store configs with multi-location placement, NPC peds/blips, buyer storefront NUI (categories, cart, buy/sell), government fund. *Deliverable: vorp_stores can be removed.*

Phase 2 — Player Ownership Core. Player store entity, admin dashboard v1 (assign owner, code, price, tax rate), stock + storage management, pricing, open/close, branding + blips, operating ledger with typed transactions.

Phase 3 — Staff. Employee roster, permission flags, hire/fire, clock system with presence verification, hourly/daily pay, NPC cashier swap-in, staff panel.

Phase 4 — Economy Automation. Buy orders, sales/discount timers, tax ledger + monthly collection + 72-hour delinquency flow + P.O. Box letters, repossession, inactivity monitor, webhook layers.

Phase 5 — Polish & Expansion. Dashboard analytics, unpaid-wage reporting, exports for the future real-estate/property script, government fund spend API.

16. Open Items

1. Does sovereign_postoffice already support system-generated mail with no sender P.O. Box (see §5 export contract)? If not, `SendSystemMail` needs to be added there before Phase 4.
2. Presence heartbeat tuning: radius, tick interval, grace period defaults.
3. Do closed player stores hide their blip entirely or dim it? (config default to pick)